

MESA · AI AND ITS APPLICATION · FIELD GUIDE

From Zero to AI Builder

What you actually built, how it works, why every tool is there, and a two-month plan to really learn AI. Written for someone starting from nothing.

Your quest map (tick a box when you can explain it to a friend)

1

What AI really is

2

How apps work

3

Your toolbox

4

The builder's playbook

5

Aangan, start to end

6

Viva + 8-week plan

Nawaz Alam · Live project: aangan-desk.vercel.app · Code: github.com/nawazalam4u/aangan-desk

Forget robots. Today's AI is mostly **software that learned patterns from huge amounts of examples**, so it can predict, sort, or create things.



AI (Artificial Intelligence)

Any computer system doing something that normally needs human judgement: recognising a face, understanding speech, writing a reply.

e.g. Google Maps predicting traffic.



Machine Learning (ML)

The way most AI is made: instead of writing rules by hand, you show the computer many examples and it learns the pattern itself.

e.g. Gmail learned what spam looks like from millions of emails.



Generative AI

AI that **creates** new things (text, images, voice, code) instead of only sorting. ChatGPT, Claude, Gemini, Midjourney.

e.g. Writing a LinkedIn post in Meera's voice (Case 1).



LLM (Large Language Model)

The engine behind chatbots. Trained on a huge library of text, it predicts the next word again and again, which turns into sentences, answers and code.

Gemini, Claude, GPT are LLMs.



Token

LLMs read and write in small chunks called tokens (roughly 3/4 of a word). You pay per token, so cost = how much you send + how much it writes.

Our Aangan analysis costs about 5 paise per turn.



Prompt

The instructions you give the model. A good prompt is like a good job description: role, goal, rules, examples, output format.

Our Vaani agent's prompt holds Nikhil's five rules.



Context window

How much text the model can "see" at once (the prompt + the conversation + documents). Anything outside it is forgotten.

We pass the whole call transcript in one go.



Hallucination

When an AI says something confident but false. It predicts plausible words, not truth. The big reason we keep rules and checks in **code**.

The AI once called an office "out of scope". Our safety net caught it.



Training vs inference

Training = the expensive months when the model learns (done by Google/Anthropic). **Inference** = using the finished model, which is what you pay for via an API.

You never trained a model; you used them.



AI agent

An AI that doesn't just answer but **acts**: follows a goal, takes steps, uses tools (book a meeting, look up data).

The Vaani receptionist is a voice agent.



Speech-to-text / text-to-speech

STT turns voice into text (the AI's "ears"); TTS turns text into a natural voice (its "mouth"). Voice agents = STT + LLM + TTS.

Vaani shows these as Ears, Brain, Mouth.



RAG and embeddings (bonus)

RAG = "look it up, then answer": the app first finds the right documents, then gives them to the LLM.

Embeddings are number-fingerprints of meaning used to find similar text.

A future Aangan could search past projects before answering.

One-line mental model: an LLM is a brilliant intern who has read the whole internet, writes fast, never gets tired, but sometimes makes things up. So you give it clear instructions (prompt), the right documents (context), and you check its work (code + humans).

Quick quiz (+20 XP)

1. Is ChatGPT "generative AI"? Why?
2. Why do we pay per token?
3. Name one hallucination our project caught.

Answers: 1) yes, it creates new text. 2) tokens are the units the model reads and writes, so compute cost scales with them. 3) An office called "out of scope", or a possession date read as a deadline.

Every web app you built has the same parts. Picture a restaurant.



WORD	PLAIN MEANING	IN YOUR PROJECT
Front end	What runs in the browser: HTML (structure), CSS (looks), JavaScript (behaviour).	The Live AI call page, dashboard, brief.
Back end	Code on a server that the browser asks to do things it can't (or shouldn't) do itself.	Python files in <code>/api</code> : rules, Telegram, HubSpot.
API	A menu of requests one program can make to another over the internet. You send a request, you get a response.	We call Gemini's API to read a call; Vaani's API to start a call.
API key	A secret password proving it's you. Leak it and others can spend your money. Never put it in public code.	Stored as hidden settings on Vercel, not in GitHub.
Environment variable	A setting kept outside the code (keys, links), read when the app runs.	<code>GEMINI_API_KEY</code> , <code>DATABASE_URL</code> ...
Webhook	"Don't call us, we'll call you": a web address another service calls when something happens.	Vaani calls us after each call; Cal.com after each booking; Telegram when a designer taps "I'll take it".
JSON	The text format apps use to exchange data: <code>{"name": "Meera", "area": "Baner"}</code> .	Every API message.
Database	Organised long-term memory. Without it, an app forgets everything when it restarts.	Neon (Postgres) stores every call.
Server / serverless	A server is a computer always running your code. Serverless = the host runs your code only when needed; you manage no computer.	Vercel runs our Python only when a request arrives.
Git & GitHub	Git = a time machine for code (every version saved). GitHub = the online home for it, and your public portfolio.	<code>github.com/nawazalam4u/aangan-desk</code>
Deploy	Put the app on the internet so anyone can open it.	Vercel redeploys on every push to GitHub.
WebRTC	The browser technology for live audio/video calls (Google Meet uses it).	How you talk to the Vaani agent without a phone line.

Why this matters for AI jobs: most "AI products" are 10% model and 90% normal software: getting data in, calling the model, checking the answer, saving it, showing it, and connecting it to tools people already use. That 90% is exactly what you practised.

If the professor asks "why X?", the answer is always: **what job does it do, and why this one over the alternative?**

TOOL	ITS JOB (ANALOGY)	WHY WE CHOSE IT
Vercel	The landlord: hosts the website and runs the back end. 🏠	Free, deploys straight from GitHub in about 30 s, runs Python "serverless" so there's no server to manage.
GitHub	The safe + portfolio: stores every version of the code. 📁	Industry standard; public repo shows your work; linked to Vercel for auto-deploys.
Neon	The pantry: a Postgres database in the cloud. 🗄️	Free tier, fast, and it sleeps when idle. Your Supabase slots were full, and you never delete old projects (portfolio rule).
Gemini (Google)	The analyst brain: reads each call and pulls out facts. 🧠	You had a key; cheap and fast (Flash-Lite, about 1 s). The code also supports Claude. We switched models when Google retired the old one.
Vaani	The voice: phone number, ears, mouth and the live receptionist agent. 🗣️	Indian voice-AI platform (Indian voices, Hindi), WebRTC test calls, webhooks with transcripts, Cal.com integration. About ₹5.50 a minute.
Telegram bot	The doorbell for designers: an instant alert with an "I'll take it" button. 🔔	Push alerts on site beat email; one-tap claim; free. Email has no claim button and needs domain setup.
HubSpot	The customer diary (CRM): one record per customer, notes, tasks, deals. 📅	Required by the course; it is where sales follow up for weeks. The dashboard measures the assistant; HubSpot manages relationships.
Cal.com	The appointment book: books the free design consultation. 📅	Free, has an API and webhooks, and its HubSpot app logs every meeting in the CRM automatically.
Python	The language of our back end. 🐍	Readable and the most common language in AI. We used only built-in libraries, so nothing extra to install.
Claude (me)	Your coding partner: planned, wrote, tested, deployed and explained. 🤝	An AI coding agent: it reads files, runs commands, uses the browser and keeps memory of your preferences between projects.

Golden rule of keys: keys live in Vercel's hidden settings and a local `.env` file that is **never** pushed to GitHub. You rotated (replaced) keys after sharing them in chat, which is exactly what professionals do.

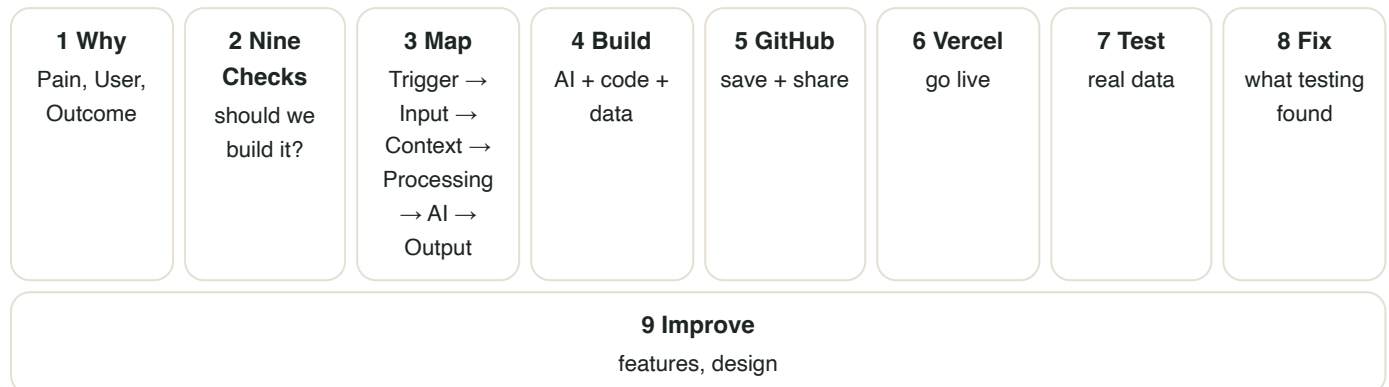
Quick quiz (+20 XP)

1. Telegram vs email: give two reasons for Telegram.
2. What is the difference between the dashboard and HubSpot?
3. Why Neon instead of Supabase this time?

Answers: 1) instant push on site + one-tap claim (measurable). 2) Dashboard = how the assistant performs; HubSpot = what happens with each customer. 3) Supabase free slots were full and you never delete old portfolio projects.

The builder's playbook (every project you did)

All your MESA projects followed the same 9 steps. Learn this once and you can explain any of them.



The Nine Checks in one breath

Kill switches (any no = don't build): 01 Problem real · 02 Workflow repeated · 03 Input available. **Sizing** (any no = build smaller): 04 Output valuable · 05 Impact measurable · 08 ROI worth it. **Boundary** (any no = a human stays in charge): 06 Failure risk OK · 07 Judgment protected · 09 Owner clear. **The Cut** = the one thing the founder wanted that a check stops you building.

Your projects so far

Case 1: Skinstinct content bot (Meera)

Meera drops rough notes into Telegram. AI scores the note, finds a news angle, and drafts a LinkedIn post in her voice; drafts are saved for APPROVE/REJECT. **Cut**: it never posts by itself; Meera publishes (Check 07).

AI role: scoring + writing. Human gate: approval.

Case 2: Kargo hiring screener

CVs are uploaded; names and contact details are removed before the AI sees them (fairness and privacy); the AI scores against a rubric and writes a brief with interview questions; emails need a human to confirm.

AI role: reading and ranking. Human gate: sending decisions.

TripLock (group trip planner)

Friends answer 5 private questions; AI suggests destinations; code scores how well each fits every friend without revealing anyone's budget; the group votes and locks the trip.

AI role: ideas. Code role: fairness and privacy.

Movie / TV matchmaker

A recommender: it takes people's tastes and picks something to watch tonight, with AI helping to match and explain the choice.

AI role: matching + explaining. Pattern: preferences in, pick out.

The pattern behind all four, and Aangan: AI does the fuzzy human part (read, understand, write). **Plain code** does the rules that must never break (privacy, fairness, prices). **A human** owns anything consequential (publishing, hiring, pricing). Say this sentence in your viva and you'll sound like an expert.

Pain

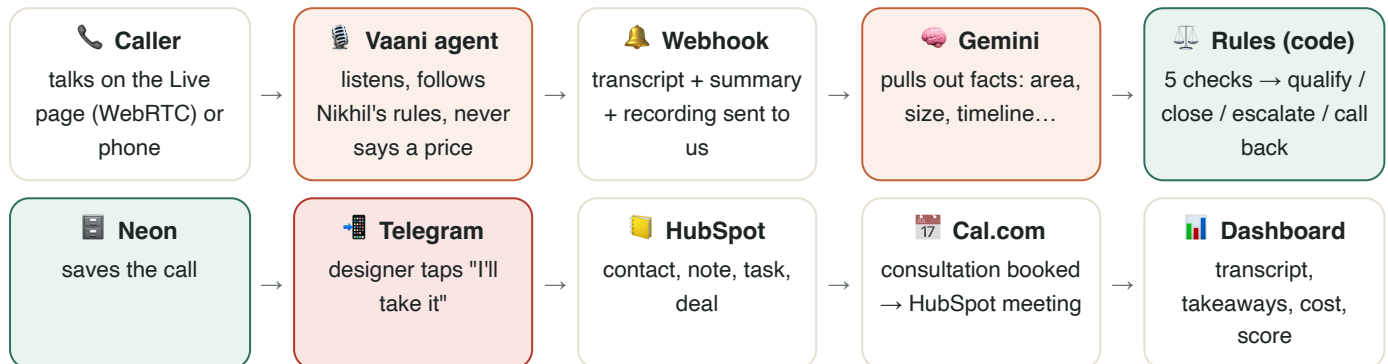
~200 enquiries a month; 48% got no reply in 48 hours; a third arrive after 7pm. Fast replies convert 4x better. Designers re-ask every question.

User

The caller (talks to the AI), the 14 designers (get the note), Nikhil (owns the rules, reads the dashboard).

Outcome

Every call answered instantly, the right ones sent to a designer with everything asked, zero prices quoted, and numbers Nikhil never pulls by hand.

How one real call travels**Nikhil's five checks (the "rubric")**

#	CHECK	PASS EXAMPLE	FAIL EXAMPLE
1	Real project (design + execution)	Full 3BHK redesign	"Just give me colour ideas", restaurant, gym
2	In Pune / PCMC	Baner, Kothrud	Nashik, Talegaon
3	Realistic timeline	Ready by March	Needed in 3 weeks (before Diwali)
4	Budget broadly OK (never asked)	Not mentioned = fine	₹1–1.5 lakh for kitchen + bedroom
5	Decision-maker on the call	"My husband and I own it"	Unclear = note it, don't push

The Cut: Nikhil wanted the AI to quote per-square-foot prices. His own pricing guide says no number may ever be quoted (prices vary by site; a wrong figure on an ₹8–14 lakh job can't be taken back). Check 06 (Failure risk) fails, so pricing stays with the designer. We enforce it twice: in the agent's prompt **and** in code that scans every transcript and alerts on Telegram if a price ever slips.

Real projects are mostly finding and fixing problems. Knowing these makes your viva answers believable.

- **Read the case.** PDF + 40 real enquiries + services, pricing and rubric files. Spotted red flags: the pricing contradiction, conflicting timeline rules, a typo ("Arjun").
- **Wrote the brain.** Rules in code from Nikhil's rubric; tested on all 40 real enquiries (40/40 agree). **Fix:** a budget check that wrongly used the whole flat size for a 2-room job.
- **Built the website and deployed on Vercel**, with a demo voice simulator in the browser.
- **Connected the real AI (Gemini).** **Problem:** the first key was cut in half when pasted; then Google retired the model. **Fix:** new key, Gemini 3.1 Flash-Lite with a backup model.
- **Real-AI testing found 2 dangerous mistakes:** an office labelled "out of scope", a possession date read as a deadline. **Fix:** clearer prompt + a safety net (the AI can't close a lead unless the caller's own words support it).
- **Neon instead of Supabase** (slots full). Telegram bot with self-registration (/register), dashboard locked with a key because it holds phone numbers.
- **HubSpot** via a service key on the school account; verified a real contact, note, task and deal.
- **Vaani voice agent** with Nikhil's rules; webhook back to our app; outbound call button.
- **WebRTC calls.** **Problem:** calls dropped after 3 s, because our call library was newer than Vaani's server. **Fix:** matched versions. To test without a human, we generated an Indian-English caller voice and fed it in as a microphone. **Problem:** Chrome's sandbox blocked that audio file; fixed for the test browser only.
- **Real calls exposed a logic bug:** a caller whose area wasn't heard was marked "qualified". **Fix:** on real calls, a missing area, project or timeline means "call back", never "qualified".
- **Vaani's real data differed from its docs** (seconds, not milliseconds; "web-user" instead of a number). **Fix:** handle both.
- **Cal.com + HubSpot.** Booking type, booking webhook, HubSpot app. **Problem:** Cal.com's firewall blocked our requests until we added a proper "user agent".
- **Final polish:** dashboard with real calls only, takeaways and decisions, a gamified "Studio Pulse", faster loading (parallel + cache + auto-refresh), magic links instead of typing keys.

Numbers to remember: 40/40 rules agreement · 39/39 with the real AI · answers in under 1 s · ~₹0.4 of AI per call · ~₹5.50 per minute of voice · 0 prices quoted in every test.

Cover the answer with your hand, say it out loud, then check.

1. What problem did you solve?

Half of Aangan's enquiries went unanswered for two days. Our AI receptionist answers every call instantly, qualifies it with the founder's rules, and hands designers a complete note.

2. Is it a real voice agent?

Yes, a Vaani voice agent. You can talk to it over WebRTC on the Live page, or it can phone a number. Real call transcripts are on the dashboard.

3. Where exactly is the AI?

Two places: the Vaani agent (listening and talking) and Gemini (reading the transcript and extracting facts). Decisions are made by plain code.

4. Why not let the AI decide?

AI can hallucinate. Code is predictable and testable: we tested it on all 40 real enquiries, and Nikhil can change a rule without touching a prompt.

5. Why no prices?

The pricing guide forbids any figure; a wrong number on an ₹8–14 lakh job can't be undone. Check 06 fails, so the designer owns pricing. Code also audits every call for prices.

6. What's a webhook here?

Vaani calls our address after each call with the transcript; Cal.com calls after each booking; Telegram calls when a designer claims a lead.

7. Why Telegram over email?

Designers are on site: a push alert is seen in seconds, and the "I'll take it" button stops double calls and measures claim time.

8. Why HubSpot?

It's the CRM: one record per customer with notes, tasks and deals for weeks of follow-up. Our dashboard measures the assistant; HubSpot manages relationships.

9. Why Cal.com?

To book the free consultation; its HubSpot app logs the meeting automatically, and its webhook marks the lead "booked" on our dashboard.

10. What does it cost?

About ₹5.50 a minute for voice plus about ₹0.4 of AI analysis per call; hosting, database and CRM are on free tiers. One extra project pays for months.

11. How did you test it?

Rules on 40 real enquiries, the real AI on 39, attack tests for prices, simulated phone and webhook tests, and real WebRTC calls with a synthetic caller voice.

12. A mistake you found?

A caller whose area wasn't heard was marked qualified. Now any real call missing area, project or timeline becomes a call-back.

13. What is WebRTC?

Browser technology for live audio (like Google Meet). It lets anyone test the agent without a phone number.

14. How is data kept safe?

Keys only in hidden settings, never in GitHub; the dashboard and live calls need a key; closed callers never enter the CRM; the greeting says the call is recorded.

15. Limits?

No inbound phone number bought yet; speech recognition can mishear names (so we ask for a call-back instead of guessing); two rules await Nikhil's confirmation.

16. How would you extend it?

WhatsApp and the web form would reuse the same rules and dashboard; a real phone number; an automatic nudge when nobody claims a lead in 15 minutes.

About 1–1.5 hours a day. Each week: learn a little, build a little, explain it to someone. All resources listed are free.

WEEK 1 What AI is (no code)

- Andrew Ng, "AI for Everyone" and "Generative AI for Everyone" (Coursera, audit free)
- 3Blue1Brown "Neural networks" videos (first 3)
- ☐ Explain ML vs generative AI to a friend

WEEK 2 Prompting like a pro

- Anthropic prompt engineering guide + interactive tutorial; Google's prompting guide
- Practice: role, goal, rules, examples, output format
- ☐ Rewrite the Aangan agent prompt in your own words

WEEK 3 Python basics

- freeCodeCamp or Kaggle "Python" (variables, lists, dicts, functions, loops)
- Read `api/_core.py`; find the five rules
- ☐ Write a script that counts words in a transcript

WEEK 4 APIs and your first AI app

- What HTTP, JSON and API keys are (MDN "HTTP overview")
- Call the Gemini or Claude API from Python with your own key
- ☐ Build: "paste a review → AI says positive/negative + why"

WEEK 5 How LLMs work inside

- Andrej Karpathy, "Intro to Large Language Models" (1-hour talk)
- Tokens, embeddings, training vs fine-tuning, temperature, context window
- ☐ Explain "next-token prediction" with an example

WEEK 6 RAG: AI that reads your documents

- Hugging Face or LangChain RAG tutorial; embeddings and vector search
- ☐ Build: ask questions about the Aangan services PDF

WEEK 7 Agents, tools and voice

- What tool use / function calling is; Anthropic "Building effective agents" article
- Revisit your Vaani agent: Ears (STT), Brain (LLM), Mouth (TTS), Actions (tools)
- ☐ Add one tool to a small agent (e.g. look up the weather)

WEEK 8 Evaluation, safety and your portfolio

- How to test AI: test sets, failure cases, guardrails, human-in-the-loop (what we did with the 40 enquiries)
- Polish GitHub READMEs; record a 2-minute demo video per project
- ☐ Write a one-page case study for Aangan Desk

Roles this prepares you for: AI product analyst or associate PM, AI solutions / implementation consultant, founder's office AI lead, automation specialist. Recruiters look for: (1) you can explain AI simply, (2) you shipped real things (you have 5 live apps), (3) you know the limits (hallucination, cost, privacy) and design around them.

Badges to earn this term



Explain AI vs ML vs GenAI in
1 minute



Draw the call flow from
memory



Explain "The Cut" without
notes



Explain API keys and why
they're secret



Change one rule in the code
yourself



Run the tests with
`python3 -I
tests/backtest.py`



Do a live WebRTC call in
front of someone



Build one tiny AI app alone
(Week 4)

Your 30-second elevator pitch

"Aangan Studio was losing half its enquiries because nobody answered in time. I built an AI voice receptionist on Vaani that answers every call instantly, qualifies callers with the founder's own rules in code, never quotes prices, and sends designers a complete note on Telegram. Leads flow into HubSpot, consultations are booked on Cal.com, and a live dashboard shows every call's transcript, decision and cost. I tested it on 40 real enquiries and real voice calls, and fixed the mistakes testing uncovered."

Where to find everything

Live AI call	aangan-desk.vercel.app/live
Dashboard	aangan-desk.vercel.app/dashboard
How it connects	aangan-desk.vercel.app/connect
Brief, Nine Checks, Cut, map	aangan-desk.vercel.app/brief
Code (public)	github.com/nawazalam4u/aangan-desk

Written by Claude (your AI coding partner) for Nawaz Alam, October 2026. The access key is deliberately not printed here. Share it only with your evaluator.